

Guia do Aluno: BigQuery Sandbox & Parquet

Carregamento, Ajustes de Sintaxe e Benchmarking (~700.000 Registros)

1. Visão Geral e Objetivo do Laboratório

Este guia orienta os alunos no carregamento e análise da base de telemetria industrial com cerca de **700.000 registros** no Google BigQuery Sandbox. O objetivo é comparar a performance de um banco relacional transacional (PostgreSQL local) com um Data Warehouse colunar em nuvem rodando sobre arquivos no formato otimizado Apache Parquet.

Resumo do Laboratório

Volume de Dados: ~700.000 linhas de telemetria de sensores industriais (2015–2021).

Formato: Apache Parquet (armazenamento colunar binário com metadados e compressão nativa).

Objetivo Prático: Medir o tempo de resposta e volume de dados lidos (Data Pruning) vs. PostgreSQL.

2. Passo 1: Configuração do BigQuery Sandbox (100% Gratuito)

O BigQuery Sandbox permite testar todas as funcionalidades do Google BigQuery sem necessidade de cadastrar cartão de crédito.

- **Acessar o Console:** Faça login em console.cloud.google.com/bigquery com sua conta Google.
- **Recursos Gratuitos:** O Sandbox oferece 10 GB de armazenamento e 1 TB de consultas mensais gratuitas.
- **Criar Projeto:** Clique no seletor de projetos no topo da tela → **Novo Projeto** (ex: *telemetria-lab*).
- **Criar Dataset:** No painel **Explorador**, clique nos 3 pontos ao lado do projeto → **Criar conjunto de dados** (ID: *telemetria_ds*, Região: *US* ou *southamerica-east1*).

3. Passo 2: Importação Direta do Arquivo Parquet

Como o arquivo Parquet de 700 mil linhas possui um tamanho otimizado (abaixo do limite de 100 MB do navegador), a importação pode ser realizada diretamente pelo console web do BigQuery:

- No painel **Explorador**, clique nos 3 pontos ao lado do dataset *telemetria_ds* → **Criar tabela**.
- **Criar tabela de:** Escolha a opção **Upload**.
- **Selecionar arquivo:** Clique em *Procurar* e selecione o arquivo *telemetria_industrial.parquet*.
- **Formato do arquivo:** Selecione **Parquet** (o esquema de colunas e tipos será detectado automaticamente pelos metadados).
- **Nome da tabela:** Digite *telemetria_industrial* e clique em **Criar tabela**.

4. Passo 3: Cuidados Essenciais de Sintaxe SQL no BigQuery

O BigQuery utiliza o dialeto Standard SQL com pequenas variações em relação ao PostgreSQL e ao DuckDB. A tabela abaixo resume os ajustes de sintaxe necessários:

Recurso / Operação	PostgreSQL / DuckDB	Google BigQuery
Identificação da Tabela	telemetria_industrial	`projeto.dataset.tabela` (com crases)
Truncamento de Data	DATE_TRUNC('day', data)	TIMESTAMP_TRUNC(data, DAY) (sem aspas)
Valores Nulos / Textos	Tratamento via NULLIF	SAFE_CAST(coluna AS FLOAT64)
Arredondamento	ROUND(val::numeric, 2)	ROUND(val, 2)

5. Passo 4: Medindo a Performance e Desativando o Cache

Por padrão, o BigQuery armazena o resultado de consultas repetidas em cache. Para realizar um teste de performance real e medir o tempo de execução do zero absoluto, siga as instruções abaixo:

- **Desativar Cache:** No editor SQL do BigQuery, clique em **Mais** (ou *Configurações*) → **Configurações de consulta** → desmarque a opção **Usar resultados em cache**.
- **Visualizar Tempo de Execução:** Após rodar a consulta, acesse a aba **Informações do job** abaixo dos resultados e observe o campo **Tempo decorrido** (*Elapsed time*).
- **Visualizar Volume Lido:** Observe o campo **Bytes processados** (*Bytes processed*). Note que o BigQuery escaneou apenas alguns MBs, provando a eficiência da leitura colunar (Data Pruning).

6. Consulta SQL de Benchmark para o BigQuery

Copie e cole a consulta abaixo no editor do BigQuery (substituindo o ID do projeto, dataset e tabela):

```
WITH telemetria_calculada AS (
  SELECT
    data,
    TIMESTAMP_TRUNC(data, DAY) AS dia,
    -- SAFE_CAST converte para número e transforma textos inválidos em NULL
    (SAFE_CAST(alimentacao_total_do_moinho_t_h AS FLOAT64) -
     SAFE_CAST(setpoint_alimentacao_do_moinho_t_h AS FLOAT64)) AS desvio_setpoint,
    -- Variação (Delta) de vibração (Window Function)
    SAFE_CAST(vibracao_do_moinho AS FLOAT64) -
    LAG(SAFE_CAST(vibracao_do_moinho AS FLOAT64), 1) OVER (ORDER BY data) AS delta_vibracao,
    -- Média móvel de 12 leituras da temperatura de saída
    AVG(SAFE_CAST(temperatura_de_saida_do_moinho_celsius AS FLOAT64)) OVER (
      ORDER BY data ROWS BETWEEN 11 PRECEDING AND CURRENT ROW
    ) AS media_movel_temp_saida,
    SAFE_CAST(potencia_do_motor_do_moinho_kw AS FLOAT64) AS potencia_motor,
    SAFE_CAST(temperatura_de_entrada_do_moinho_celsius AS FLOAT64) AS temp_entrada,
    SAFE_CAST(pressao_diferencial_do_moinho_mpa AS FLOAT64) AS pressao_diferencial,
    SAFE_CAST(potencia_do_exaustor_principal_kw AS FLOAT64) AS potencia_exaustor,
    SAFE_CAST(fluxo_de_ar_exaustor_principal_m3_h AS FLOAT64) AS fluxo_ar,
    SAFE_CAST(silo_argical_percentual AS FLOAT64) AS silo_argical,
    SAFE_CAST(silo_minerio_ferro_percentual AS FLOAT64) AS silo_minerio
  FROM `seu-projeto.telemetria_ds.telemetria_industrial`
)
```

```
SELECT
  dia,
  COUNT(*) AS total_amostras_dia,
  ROUND(AVG(desvio_setpoint), 4) AS media_desvio_setpoint,
  ROUND(STDDEV(desvio_setpoint), 4) AS std_desvio_setpoint,
  ROUND(AVG(delta_vibracao), 4) AS media_delta_vibracao,
  ROUND(MAX(delta_vibracao), 4) AS max_pico_vibracao,
  ROUND(AVG(media_movel_temp_saida), 2) AS media_temp_saida_filtrada,
  ROUND(AVG(temp_entrada), 2) AS media_temp_entrada,
  ROUND(AVG(pressao_diferencial), 4) AS media_pressao_diferencial,
  ROUND(AVG(potencia_motor), 2) AS media_potencia_moinho,
  ROUND(AVG(potencia_exaustor), 2) AS media_potencia_exaustor,
  ROUND(AVG(fluxo_ar), 2) AS media_fluxo_ar,
  ROUND(AVG(silo_argical), 2) AS media_silo_argical,
  ROUND(AVG(silo_minerio), 2) AS media_silo_minerio
FROM telemetria_calculada
GROUP BY dia
ORDER BY dia;
```